

Perypetie trenowania modeli YOLO

Historia zmagañ z przygotowaniem detekcji ludzi na potrzeby misji zawodów SUAS 2026 w kole naukowym AGH Solar Plane

Bartosz Królik

AGH Solar Plane, Akademia Górniczo-Hutnicza w Krakowie

Z wysokości ponad 60 metrów człowiek wygląda jak niewielka plamka złożona z kilkunastu pikseli. Dla człowieka patrzącego na pełne zdjęcie odnalezienie jej potrafi być trudne. Dla modelu sztucznej inteligencji sytuacja wygląda jeszcze gorzej.

Właśnie z takim problemem mierzyłem się w AGH Solar Plane podczas przygotowań do zawodów SUAS 2026. Jednym z elementów misji Search and Rescue było przeszukanie wyznaczonego obszaru, odnalezienie celów i dostarczenie w ich pobliżu odpowiednich ładunków. Zawody łączą autonomiczny lot, mapowanie terenu, lokalizację obiektów i zrzut [1]. Minimalna wysokość wynosiła 150 stóp. Jednym z elementów tej układanki było wykrycie manekina, który miał symulować zaginionego w terenie po przejściu tornada. Jak komputer może wykryć takiego manekina?

Do jego wykrywania wykorzystałem YOLO, rodzinę modeli opartych na konwolucyjnych sieciach neuronowych, trenowanych z użyciem biblioteki Ultralytics [2]. Nazwa jest skrótem od *You Only Look Once*, czyli model analizuje obraz i za jednym razem wskazuje, co rozpoznał oraz gdzie się to znajduje. Wynikiem są prostokątne ramki otaczające wykryte obiekty (ang. bounding box) i pewność detekcji (ang. confidence).



Logo zawodów SUAS: Miały one miejsce 14-17.09.2026 w miejscowości Tulsa w stanie Oklahoma.

Kiedy człowiek znika

Pierwsze eksperymenty przeprowadziłem na zbiorze NOMAD. Jest to duży zestaw zdjęć wykonanych z drona, obejmujący ponad 42 tysiące klatek, stu różnych aktorów, pięć wysokości lotu (10 m, 30 m, 50 m, 70 m, 90 m) oraz 10 poziomów zasłonięcia sylwetki [3]. Aktorzy stali, chodzili, leżeli i chowali się za przeszkodami. Na papierze wyglądało to niemal idealnie: dużo danych, realistyczne otoczenie i ludzie fotografowani z góry.

W pierwszym podejściu wykorzystałem około jednej dziesiątej zbioru. Duże obrazy zostały zmniejszone do rozmiaru obsługiwanego przez model, a następnie posłużyły do wytrenowania YOLOv8s. Rezultat był słaby: precision, czyli udział poprawnych wskazań wśród wszystkich detekcji, wyniósł około 0.48, a recall, określający, jaką część obecnych na zdjęciach ludzi rzeczywiście udało się znaleźć, tylko około 0.22.

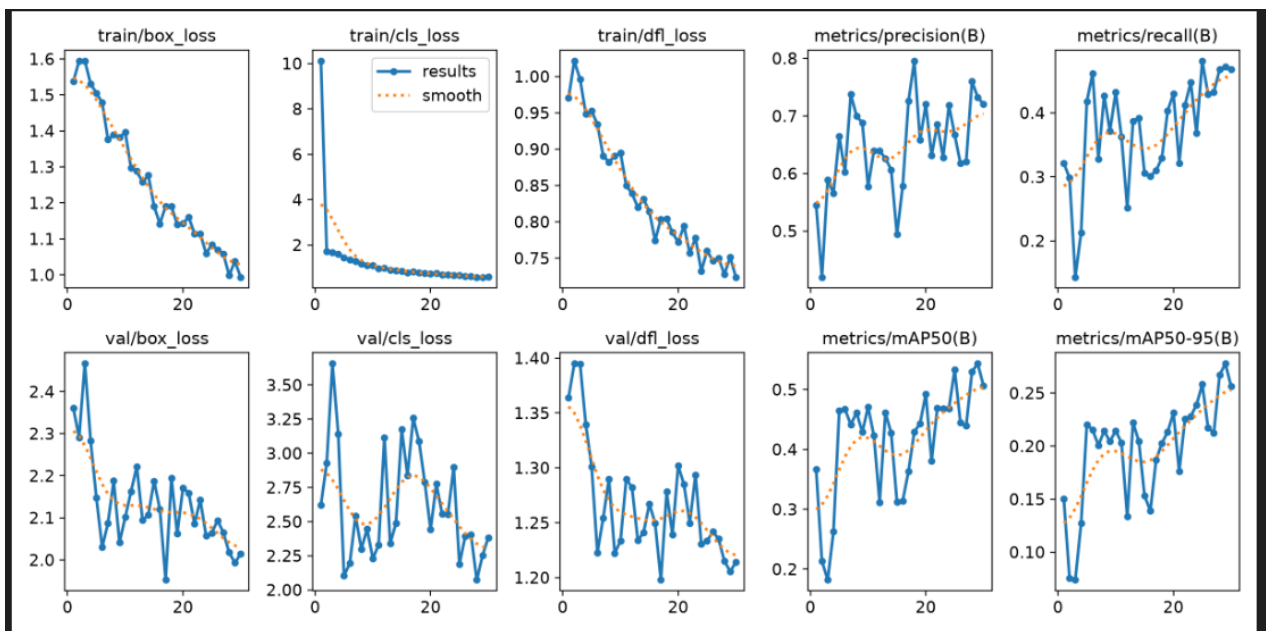
Oryginalne obrazy były rozmiaru 5.4k, natomiast model otrzymywał ich pomniejszoną wersję do 1280 x 1280 pikseli. W czasie skalowania człowiek, który już wcześniej zajmował niewielki fragment fotografii, stawał się kilkupikselową plamką. Część informacji po prostu znikiała. Trudno oczekiwać, by nawet dobry model odnalazł szczegóły, których nie ma już w obrazie wejściowym.

Rozwiązaniem okazało się cięcie zdjęć na mniejsze fragmenty. Zamiast zmniejszać całą fotografię, dzieliłem ją na kafelki. Dzięki temu znajdująca się na kafelku osoba zajmowała maksymalną liczbę pikseli, co dawało modelowi materiał, na którym mógł się uczyć. Po zastosowaniu tej metody precision wzrosło do około 0.73, a recall do 0.47. Dłuższy trening pozwolił uzyskać wartości około 0.81 i 0.54. Nadal daleko było do niezawodnego systemu, ale kierunek był wyraźny: sposób przygotowania danych treningowych był kluczowy.

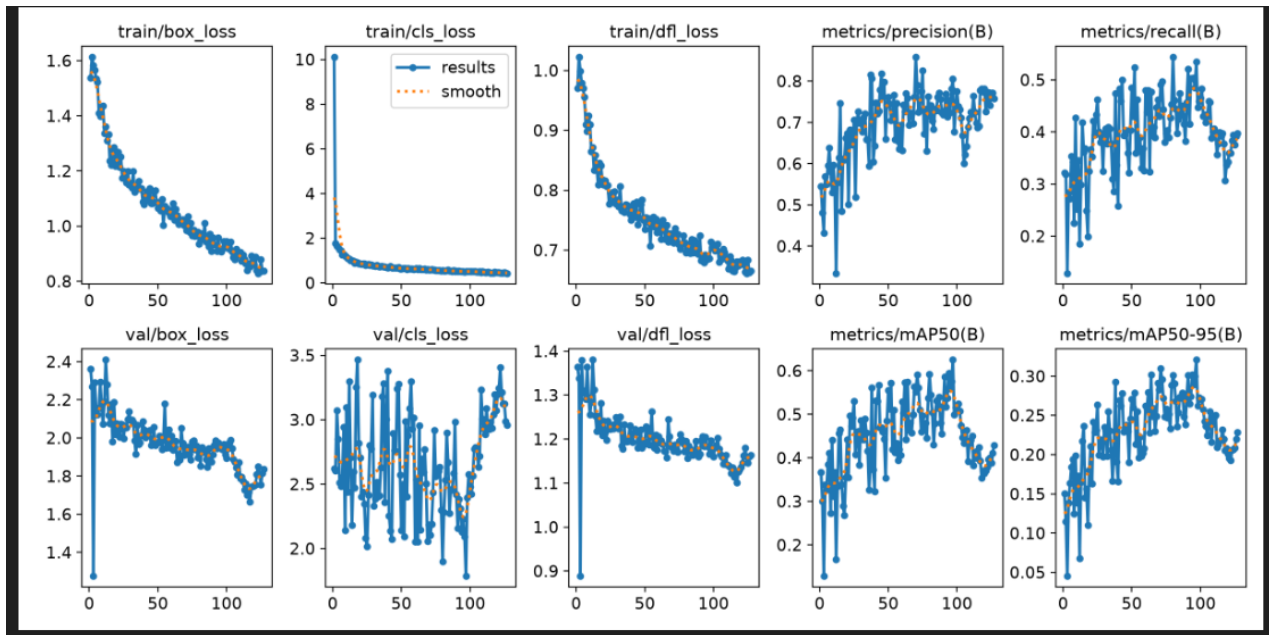
Cięcie wprowadziło też własne problemy. Regularne przesuwanie okna po obrazie mogło sprawić, że człowiek znalazł się na krawędzi kafelka albo został przecięty na pół. Było to szczególnie szkodliwe w NOMAD, gdzie część sylwetek była dodatkowo zasłonięta przez drzewa, krzewy lub elementy zabudowy. Dlatego późniejsza wersja skryptu wybierała fragment obrazu względem istniejącej adnotacji, tak aby człowiek nie został przypadkowo ucięty.



Przykładowe zdjęcie z NOMAD-a z wysokości 50m.



Wyniki pierwszego podejścia z użyciem cięcia: train dotyczy danych treningowych, val validacyjnej części zbioru, box_loss – błąd położenia ramek, mówi jak bardzo model nie trafił, cls_loss – błąd klasyfikacji, df_l_loss – dodatkowa miara precyzji położenia ramki, precision – ile wskazań było poprawnych, recall – ile rzeczywistych ludzi zostało wykrytych, mAP50 – łączona ocena jakości detekcji, mAP50-95 – bardziej wymagająca niż mAP50, wymaga dokładniejszego dopasowania ramek.



Wyniki dłuższego treningu przy użyciu cięcia: Model osiągnął najlepsze rezultaty na 97 epoce. Epoka to jeden cykl treningu, podczas którego cały zbiór danych treningowych jest „mielony” przez model tak by dopasować wagi pod lepszy wynik na zbiorze walidacyjnym.



Przykładowy batch treningowy: model dostawał zdjęcia z użyciem augmentations, czyli operacji, które miały zwiększyć różnorodność zbioru, na przykład zlepiania 4 zdjęć w jedno. Niebieskie prostokąty oznaczają ludzi (klasa o numerze 0 była do nich przypisana).

Dobry wynik, który niczego nie gwarantuje

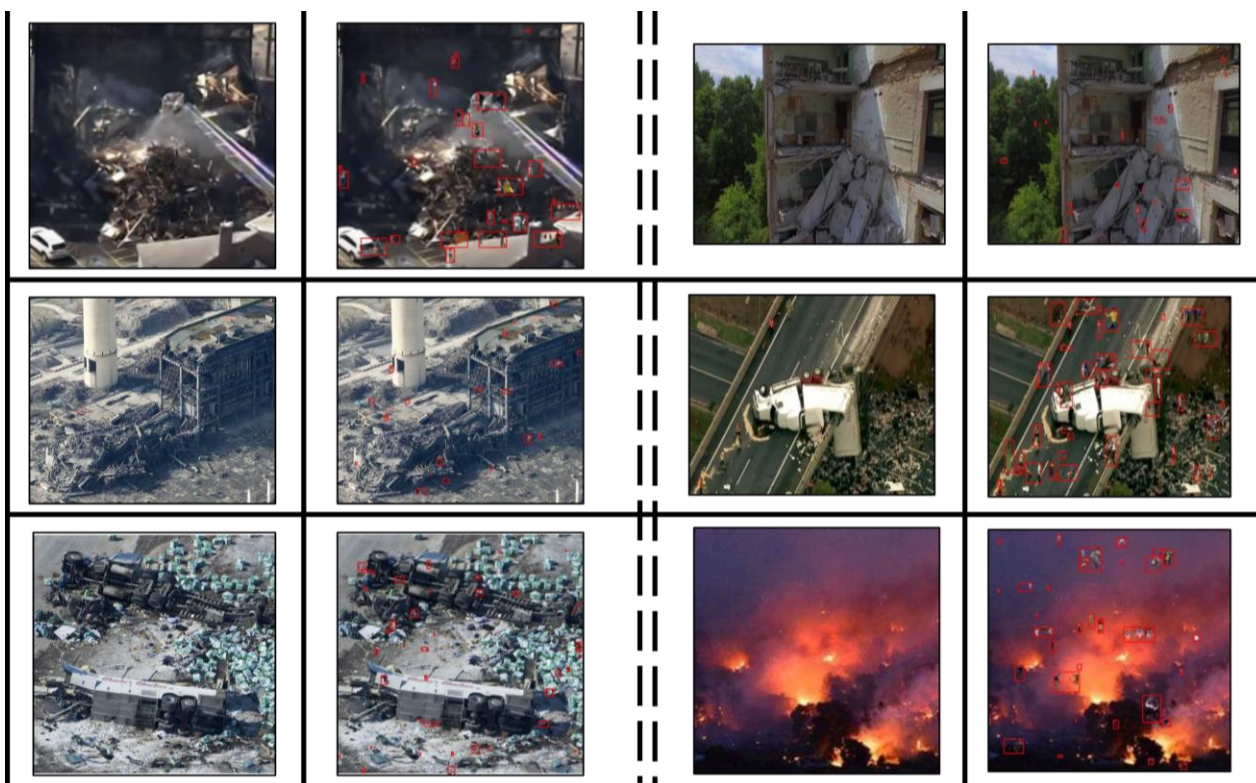
Kolejny etap polegał na sprawdzaniu różnych zbiorów danych. Oprócz NOMAD korzystałem między innymi z C2A i SARD. C2A jest zbiorem syntetycznym, w którym obrazy ludzi umieszczono na tłach przedstawiających obszary dotknięte katastrofami [4]. SARD zawiera natomiast zdjęcia aktorów odgrywających osoby wyczerpane lub poszkodowane [5]. Oba zbiory wydawały się dobrze pasować do scenariusza Search and Rescue.

Najpierw wytrenowałem YOLO11s na C2A, a później dotrenowałem go na SARD. Na testowej części tych danych model osiągnął świetne rezultaty: precision około 0.96, recall 0.86 i mAP50 około 0.93. Można byłoby uznać, że problem został praktycznie rozwiązany.

Wystarczył jednak test na NOMAD, aby ten optymizm zniknął. Precision spadło do około 0.17, recall do 0.13, a mAP50 do zaledwie 0.058. Model, który chwilę wcześniej niemal bezbłędnie znajdował ludzi, nagle zaczął mylić ich z drzewami i fragmentami terenu, a większości prawdziwych sylwetek w ogóle nie zauważał.

To przykład tak zwanego *domain gap*, czyli różnicy pomiędzy środowiskiem, na którym model się uczył, a tym, w którym próbujemy go później użyć. W SARD ludzie często byli stosunkowo duzi, dobrze widoczni i łatwi do rozpoznania. W NOMAD stawali się bardzo mali, częściowo zasłonięci i fotografowani w innym otoczeniu. Z perspektywy człowieka oba zbiory pokazują „osoby widziane z drona”. Z perspektywy sieci neuronowej są to jednak dwa różne światy.

Ta próba nauczyła mnie ważnej rzeczy: wysoki wynik walidacyjny nie jest uniwersalną oceną inteligencji modelu. Mówi jedynie, jak dobrze model radzi sobie z konkretnym zbiorem testowym. Jeżeli ten zbiór jest podobny do treningowego, wynik może wyglądać znakomicie, nawet gdy rozwiązanie słabo działa poza nim.



Przykładowe zdjęcia z C2A: wyraźnie widać, że syntetyczne stworzenie datasetu prowadzi do mało naturalnego rozmieszczenia ludzi.



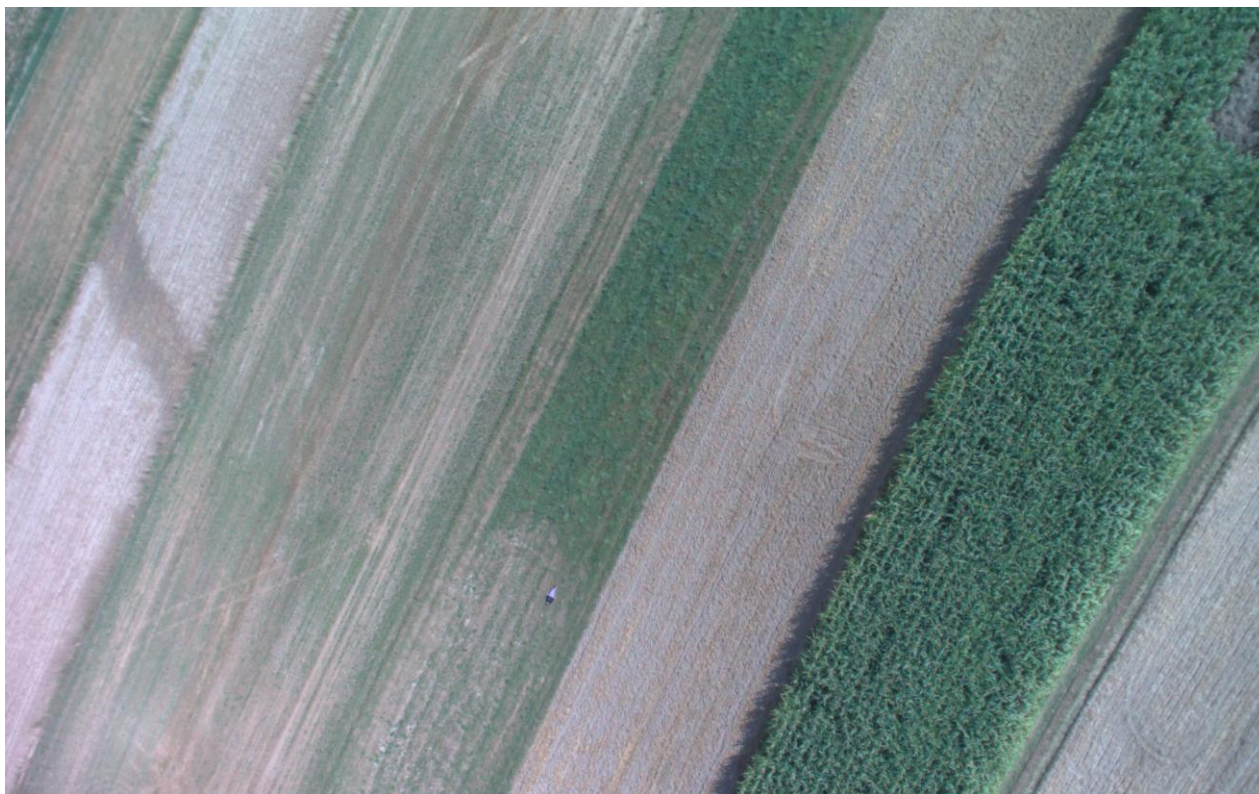
Przykładowe zdjęcie z zbioru SARD z ramkami: aktorzy są tutaj dużo lepiej widoczni i stanowią mniejszy problem dla modelu.

A potem przyszły zdjęcia z testów

Nawet model radzący sobie z NOMAD nie musiał działać dobrze na zdjęciach z naszych testów. Zmieniało się właściwie wszystko: kamera, wysokość, charakter terenu, perspektywa, kolorystyka, a przede wszystkim rozmiar celu. Na zdjęciach z naszej kamery sylwetki ludzi miały 35 x 12 pikseli, to niewiele więcej niż YOLO jest w stanie minimalnie wykryć. W porównaniu do NOMADA-u okazało się, że rozmiar aktorów zgadza się przy zdjęciach z 90 m, nasze testowe były robione z 60 m.

To drugi poziom domain gap. Pierwszy ujawnił się przy przenoszeniu modelu pomiędzy publicznymi zbiorami. Drugi pomiędzy publicznymi zbiorami, a rzeczywistymi obrazami z naszej platformy. Model mógł nauczyć się określonej skali ludzi, typu roślinności, sposobu fotografowania albo nawet charakterystycznej kompresji zdjęć. Żadna z tych cech nie musiała wystąpić podczas prawdziwego lotu.

W praktyce okazało się więc, że samo dodawanie kolejnych tysięcy cudzych obrazów niekoniecznie przybliży nas do celu. Duży zbiór danych jest wartościowy tylko wtedy, gdy zawiera informację potrzebną w docelowym zadaniu. Czasami kilkaset obrazów z właściwej kamery i wysokości może być bardziej użyteczne niż dziesiątki tysięcy zdjęć pochodzących z innego środowiska.



***Przykładowe zdjęcie z testów:** w tej skali model całkowicie zawiódł i nie był w stanie nic wykryć.*

Mieszanka danych prawdziwych i syntetycznych

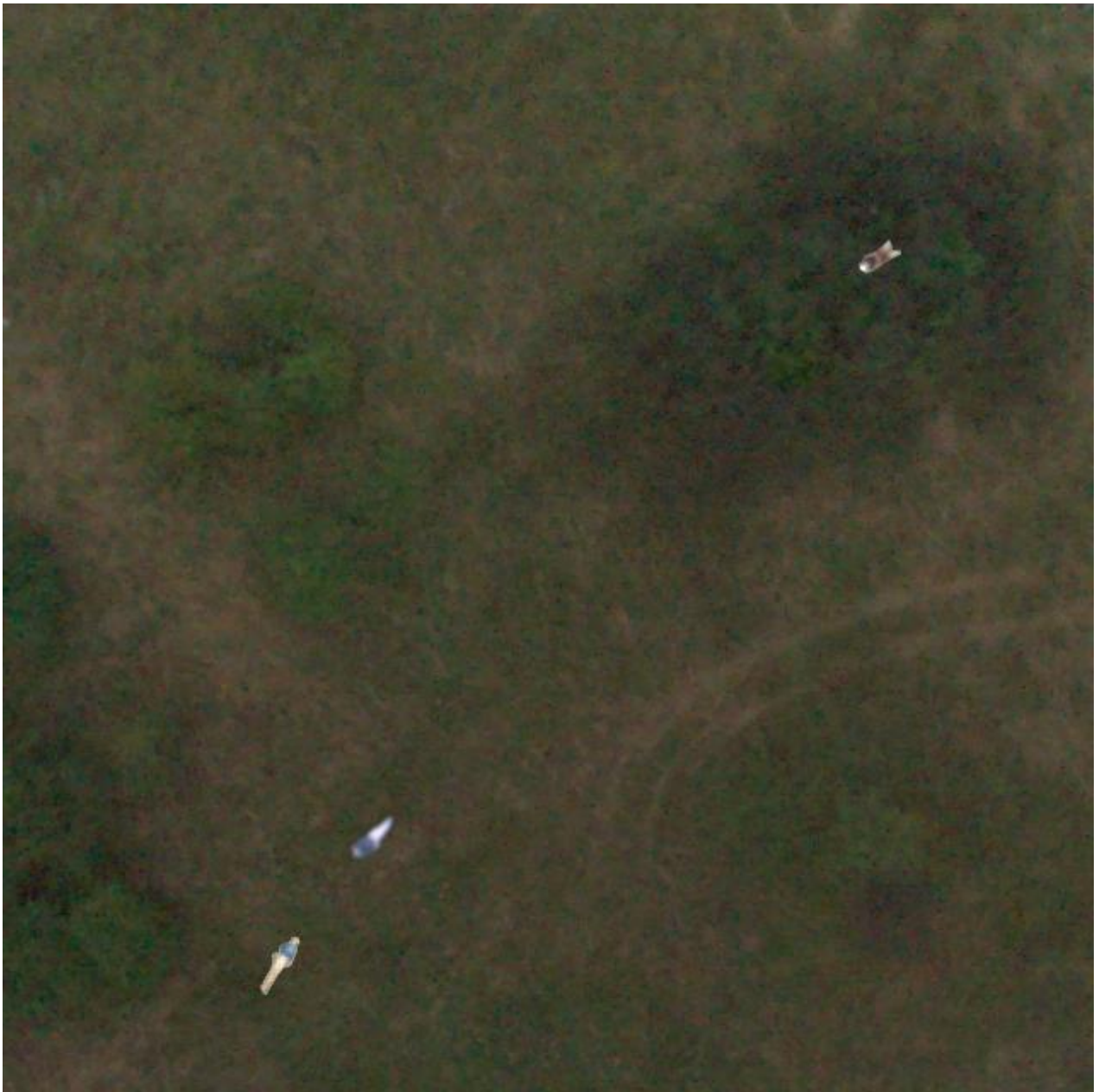
Model trzeba było wytrenować na danych bardziej zbliżonych do misji. Wykorzystałem zdjęcia zrobione podczas testów, z nich wyciąłem fragmenty przedstawiające tylko sylwetki ludzi, które następnie przy pomocy skryptu w Pythonie i biblioteki OpenCV wklejałem na różnorodne tła. Pozwalało to stosunkowo szybko tworzyć nowe przykłady, zmieniać położenie obiektu i zwiększać różnorodność zbioru.

Nie jest to rozwiązanie pozbawione wad. Wklejony obiekt może mieć inne oświetlenie niż tło, zbyt ostrą krawędź albo nienaturalną skalę. Jeżeli liczba źródłowych wycinków jest niewielka, model może nauczyć się rozpoznawać konkretne sylwetki lub artefakty montażu zamiast ogólnej cechy poszukiwanego celu. Dane syntetyczne trzeba więc traktować jako narzędzie, a nie automatyczny sposób na stworzenie dowolnie dużego, doskonałego zbioru. Szczególnie generowanie większej ilości syntetycznych zdjęć z tej samej bazy początkowej szybko przestaje przynosić rezultaty.

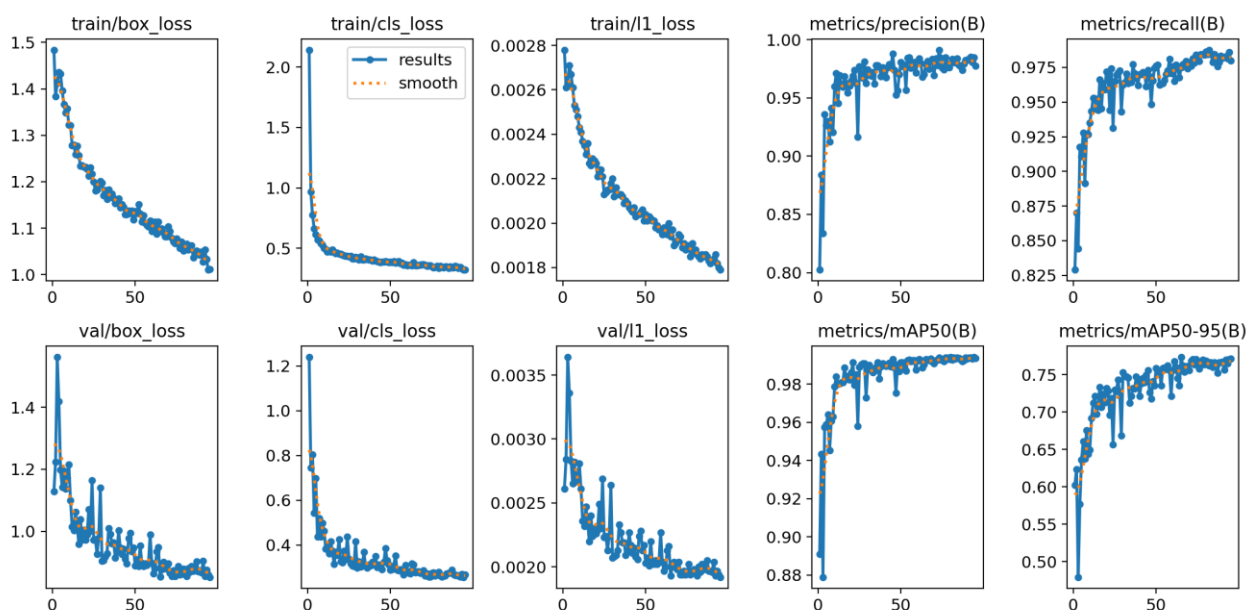
Mimo tych ograniczeń takie obrazy miały jedną ogromną zaletę: były przygotowane pod konkretny problem. Jasno pokazywałem modelowi jak wyglądają ludzie na zdjęciach z naszej kamery, w warunkach zbliżonych do zawodów. Największy problem stanowiło wyuczenie modelu dokładnie wyciętych sylwetek (ang. overfitting). Model wtedy bezbłędnie radzi sobie z danymi treningowymi, ale nie potrafi generalizować i zawodzi, gdy manekin będzie trochę inny.

Podczas treningów korzystałem głównie z Kaggle. Platforma udostępnia środowiska notebookowe z akceleracją GPU i pozwala uruchamiać długie eksperymenty bez blokowania własnego komputera [6]. Limit czasu pojedynczej sesji wielokrotnie przypominał mi jednak, że plan treningu trzeba dobrze przemyśleć. Testowałem różne warianty modeli. Większy model potrafił poprawiać wyniki, ale nie usuwał problemów danych. Jeśli zbiór był źle dopasowany albo podzielony, dodatkowe miliony parametrów nie stanowiły magicznego rozwiązania.

Duże zdjęcia podczas inferencji nadal wymagały dzielenia. Wykorzystałem do tego SAHI, otwartą metodę predykcji kafelkowej opracowaną z myślą o wykrywaniu małych obiektów na obrazach o wysokiej rozdzielczości [7, 8]. Obraz był cięty na nakładające się fragmenty o rozmiarze 640×640 pikseli, model analizował każdy z nich, a wyniki były następnie składane z powrotem w jedną całość. Nakładanie fragmentów ograniczało ryzyko, że człowiek znajdzie się dokładnie na granicy dwóch kafelków. Ceną był dłuższy czas obliczeń, który był istotny, ponieważ docelowo system miał pracować na Jetsonie Orin Nano, a nie na serwerze z potężną kartą graficzną.



Przykładowe syntetyczne zdjęcie: trzy sylwetki wklejone w tło, slice 640x640px.

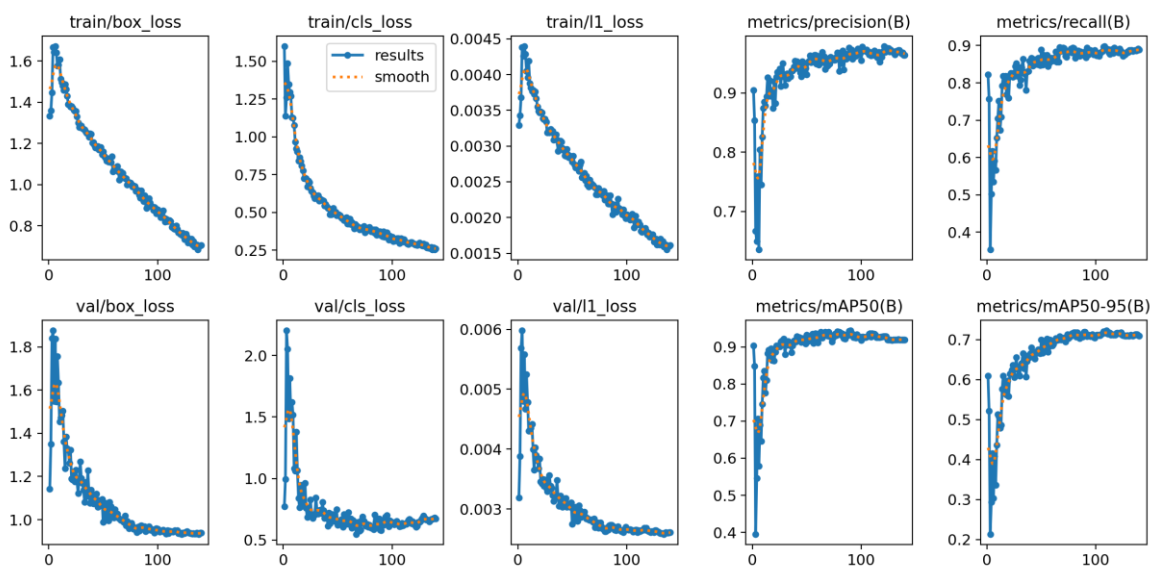


***Rezultaty treningu na samych wycinkach:** model bardzo dobrze wyuczył się rozpoznawać dane treningowe, ale testy na zdjęciach z NOMAD-a pokazały, że nie generalizuje.*

Wszystkie ręce na pokład – ostateczny dataset

Zawody zbliżały się wielkimi krokami, a testy terenowe pokazały, że model wytrenowany tylko na zbiorze NOMAD zdecydowanie sobie nie radzi. Z drugiej strony wycinków sylwetek z testów, które były unikalne, było ledwie kilkadziesiąt. Dataset z nich prowadził do overfittingu i słabej generalizacji. Stąd wziął się pomysł na połączenie źródeł danych. Pierwszą część stanowiły wyselekcjonowane zdjęcia z NOMAD-a. Wziąłem tylko wysokości 70 m i 90 m oraz odpuściłem najbardziej zakryte zdjęcia. Z wycinków sylwetek z testów stworzyłem liczący 4000 zdjęć syntetyczny dataset. Zbiór został dopełniony syntetycznymi datasetami, do których generacji zostały użyte modele AI (GPT 6 Astra oraz Gemini 3.1 pro). Przy generowaniu i labelowaniu (dodawaniu adnotacji) nieocenione była pomoc zespołu drona.

Trening przyniósł dobre rezultaty, model na pewno umiał rozpoznawać ludzi na danych treningowych:



Rezultaty treningu na mieszanym zbiorze: model osiągnął wysokie wyniki, co dawało nadzieję na lepszy przebieg testów.

Testy terenowe pokazały, że model działa, ale na pewno nie jest idealny. Przez większość czasu wykrywał ludzi i dawał tym detekcjom wysoki confidence, ale potrafił identyfikować też trawę, gałęzie czy kamienie jako osoby. Niemniej jednak, sumarycznie wyniki detekcji dawały jasny obraz, gdzie jest człowiek. Na locie testowym liczba klatek, w których faktyczne wykrycie było robione, była liczona w setkach, a przypadkowe obiekty miały takich detekcji co najwyżej kilkadziesiąt.



Rezultaty z testów: został dodany także dataset do namiotów, by robić detekcję obu celów na SUAS 2026 jednocześnie.

Zawody SUAS 2026

Model wytrenowany na mieszanym zbiorze dał radę wykryć manekina. A oprogramowanie i mechanizm zrzutu zadziały bezbłędnie dostarczając butelkę wody.

Ostatecznie AGH Solar Plane zakończył rywalizację SUAS 2026 na piątym miejscu w klasyfikacji Mission Demonstration [9]. Opis platformy Świerszcz i prac nad jej systemami znajduje się na stronie zespołu [10], a szersze informacje o działalności koła na stronie głównej AGH Solar Plane [11].

Najważniejszy wniosek z tej historii jest prosty: w uczeniu maszynowym model stanowi tylko część rozwiązania. Równie ważne są dane, sposób ich przygotowania, uczciwy podział na trening i test oraz sprawdzenie działania w środowisku, do którego system został zaprojektowany. Wynik na wykresie może wyglądać imponująco, lecz dopiero obraz z prawdziwego lotu pokazuje, czego model nauczył się naprawdę.

Historia przygotowań detekcji do SUAS była więc nie tyle opowieścią o znalezieniu idealnej wersji YOLO, ile o stopniowym odkrywaniu, gdzie naprawdę znajduje się problem. Czasem był nim model. Częściej jednak człowiek, którego próbowałem znaleźć, stawał się zbyt mały, trafiał do niewłaściwego kafelka albo wyglądał zupełnie inaczej niż ludzie pokazani sieci podczas treningu. Dopiero gdy zacząłem patrzeć na dane z perspektywy rzeczywistej misji, kolejne wyniki zaczęły mieć praktyczne znaczenie.

Bibliografia i materiały internetowe

- [1] RoboNation, *Student Unmanned Aerial Systems Competition*, oficjalna strona zawodów SUAS. <https://suas-competition.org/>
- [2] Ultralytics, *Model Training with Ultralytics YOLO*, dokumentacja treningu modeli YOLO. <https://docs.ultralytics.com/modes/train/>
- [3] A. M. Russell Bernal, W. Scheirer, J. Cleland-Huang, *NOMAD: A Natural, Occluded, Multi-Scale Aerial Dataset, for Emergency Response Scenarios*, WACV 2024, s. 8584–8595. https://openaccess.thecvf.com/content/WACV2024/html/Bernal_NOMAD_A_Natural_Occluded_Multi-Scale_Aerial_Dataset_for_Emergency_Response_WACV_2024_paper.html
- [4] R. A. Nihal, B. Yen, K. Itoyama, K. Nakadai, *UAV-Enhanced Combination to Application: Comprehensive Analysis and Benchmarking of a Human Detection Dataset for Disaster Scenarios*, 2024. <https://arxiv.org/abs/2408.04922>
- [5] S. Sambolek, M. Ivašić-Kos, *Search and Rescue Image Dataset for Person Detection – SARD*, IEEE DataPort, 2021. <https://doi.org/10.21227/ahxm-k331>

- [6] Kaggle, *Getting Started on Kaggle Notebooks*, dokumentacja środowiska notebookowego i akceleracji GPU. <https://www.kaggle.com/docs/notebooks>
- [7] F. C. Akyon, S. O. Altinuc, A. Temizel, *Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection*, 2022. <https://arxiv.org/abs/2202.06934>
- [8] Ultralytics, *YOLO26 Sliced Inference with SAHI*, dokumentacja predykcji kafelkowej. <https://docs.ultralytics.com/guides/sahi-tiled-inference/>
- [9] RoboNation, *2026 SUAS Competition Final Standings*, wyniki zawodów SUAS 2026. <https://suas-competition.org/2026/results>
- [10] AGH Solar Plane, Świerszcz, opis platformy przygotowanej do misji SUAS 2026. <https://www.aghsolarplane.com/pl/projects/swierszcz>
- [11] AGH Solar Plane, oficjalna strona koła naukowego. <https://www.aghsolarplane.com/>